

Reviewer Pack — Kashin-Split Energy Gap Lower-Bounds Sign-Matrix Rigidity at...

Entry 223537333300 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 12:12:14 UTC

Kashin-Split Energy Gap Lower-Bounds Sign-Matrix Rigidity at Rank $n/4$

Entry ID: 223537333300 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 12:12:14 UTC

1. Conjecture

Field A (mathematical branch): Discrepancy / dispersion via Kashin-style orthogonal decompositions (Kashin's theorem on splitting $\ell_2^{\{2n\}}$ into nearly ℓ_1 -equivalent halves; Bourgain-Tzafriri restricted invertibility flavor) **Field B** (complexity object): Matrix rigidity (Valiant) of $n \times n \{+1, -1\}$ sign matrices at target rank $r = n/4$, measured as the minimum Hamming weight of a perturbation E such that $\text{rank}(M+E) \leq r$

Statement:

For every $n \times n \{+1, -1\}$ matrix M , partition the n columns greedily into two halves (A, B) maximizing the Frobenius energy gap $g(M) = ||M_A||_F^2 - ||M_B||_F^2 \mid / n^2$, and let $s_{\text{low}}(M)$ be the sum of the bottom $3n/4$ singular values of M divided by $n^{\{3/2\}}$. Then the rigidity at rank $n/4$ satisfies $R_M(n/4) \geq c * n^2 * (s_{\text{low}}(M) - g(M)) +$ for an absolute constant $c > 0$ (we conjecture $c = 1/16$). Equivalently: whenever a sign matrix has a small Kashin energy-gap (g small) but a large bottom-tail singular sum, it must be rigid.

Rationale (proposer's reasoning):

Kashin splittings expose the obstruction that prevents low-rank approximation from cheaply absorbing many sign-flips: when no column-bipartition concentrates Frobenius energy, the residual after removing the top $n/4$ spectrum cannot be packed into few sparse perturbations without violating a discrepancy-style dispersion inequality of Bourgain-Tzafriri type. This bridges the dispersion side of discrepancy theory (energy distribution under random orthogonal partitions) with Valiant's rigidity, an angle not pursued in the existing rigidity literature which focuses on either spectral lower bounds (Lokam) or paving (Alon-Panigrahy-Yekhanin).

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 44acebec51fa75b7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across n in $\{6,8,10,12,14,16\}$, for each sampled matrix compute $\delta_M = R_M(n/4)/n^2 - (1/16)(s_{\text{low}}(M) - g(M))_+$. Conjecture is SUPPORTED iff $\geq 95\%$ of all instances (≥ 200 random + structured per n , ≥ 1200 total) satisfy $\delta_M \geq 0$ AND mean $\delta_M \geq 0$ per n . FALSIFIED if any single instance has $\delta_M < -0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - matrix rigidity sign matrix Hadamard singular values rank perturbation - Kashin decomposition restricted invertibility Bourgain-Tzafriri singular value lower bound - matrix rigidity Valiant lower bound spectral discrepancy sign matrix $n/4$

Top relevant hits considered: - [<http://arxiv.org/abs/math/0610498v4>] Bounds on changes in Ritz values for a perturbed invariant subspace of a Hermitian matrix - [<http://arxiv.org/abs/1005.0979v1>] Supersymmetry in Random Matrix Theory - [<http://arxiv.org/abs/1204.0062v4>] Improved matrix algorithms via the Subsampled Randomized Hadamard Transform - [<http://arxiv.org/abs/2208.05223v1>] Modular Bourgain-Tzafriri Restricted Invertibility Conjectures and Johnson-Lindenstrauss Flattening Conjecture - [<http://arxiv.org/abs/1503.07054v1>] A Geometric Perspective on the Singular Value Decomposition - [<http://arxiv.org/abs/2503.20972v3>] p-adic Grothendieck Inequality, p-adic Johnson-Lindenstrauss Flattening and p-adic Bourgain-Tzafriri Restricted Invertibility - [<http://arxiv.org/abs/1210.1461v1>] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def frobenius_norm(M):
        return sum(sum(x**2 for x in row) for row in M)**0.5

    def singular_values(M):
        n = len(M)
        U = []
        S = []
```

```

V = []

# QR decomposition
A = M.copy()
Q = []
R = []
for i in range(n):
    q = [A[j][i] for j in range(i, n)]
    norm_q = frobenius_norm(q)
    u = [x / norm_q for x in q]
    Q.append(u)
    r = [sum(A[j][k] * u[k] for k in range(k)) for k in range(n)]
    R.append(r)

# SVD from QR
U = Q
V = Q
S = [R[i][i] for i in range(n)]

return U, S, V

def frobenius_energy_gap(M):
    n = len(M)
    columns = list(range(n))
    best_gap = 0

    # Exhaustive search for column bipartition
    if n <= 14:
        for mask in range(1 << (n - 1)):
            A = [columns[i] for i in range(n) if mask & (1 << i)]
            B = [columns[i] for i in range(n) if not (mask & (1 << i))]
            M_A = [[M[i][j] for j in A] for i in range(n)]
            M_B = [[M[i][j] for j in B] for i in range(n)]
            gap = abs(sum(x**2 for row in M_A for x in row) - sum(x**2 for row in M_B for x in row))
            if gap > best_gap:
                best_gap = gap

    # Local search for column bipartition (n=16)
    else:
        for _ in range(1000):
            A = random.sample(columns, n // 2)
            B = [col for col in columns if col not in A]
            M_A = [[M[i][j] for j in A] for i in range(n)]
            M_B = [[M[i][j] for j in B] for i in range(n)]
            gap = abs(sum(x**2 for row in M_A for x in row) - sum(x**2 for row in M_B for x in row))
            if gap > best_gap:
                best_gap = gap

    return best_gap

def rigidity(M, rank):
    n = len(M)
    k = 0
    while True:
        perturbation = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(rank)]
        M_perturbed = [[M[i][j] + perturbation[i][j] for j in range(n)] for i in range(n)]

```

```

        U, S, V = singular_values(M_perturbed)
        if sum(S[:rank]) == n:
            k += 1
        else:
            break

    return k

def s_low(M):
    _, S, _ = singular_values(M)
    return sum(sorted(S)[-3*n//4:]) / (n**(3/2))

n = random.choice([6, 8, 10, 12, 14, 16])
M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]

g_M = frobenius_energy_gap(M)
s_low_M = s_low(M)
R_M_n4 = rigidity(M, n // 4)

delta_M = R_M_n4 / n**2 - (1/16) * max(s_low_M - g_M, 0)

return {
    "metric_name": "delta_M",
    "metric_value": delta_M,
    "instances_tested": 1,
    "conjecture_holds": delta_M >= 0,
    "counterexample": "" if delta_M >= 0 else f"Rigidity {R_M_n4} is less than expected by {(1/16) * (s_low_M - g_M)}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    total_delta_M = 0
    num_instances = 0

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")

        if result["conjecture_holds"]:
            total_delta_M += result["metric_value"]
            num_instances += result["instances_tested"]

    mean_delta_M = total_delta_M / num_instances
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results) or support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_delta_M} std=0 support_fraction={support_fraction}")
    elif any(r["delta_M"] < -0.05 for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["delta_M"] < -0.05)
        print(f"RESULT: FALSIFIED counterexample=\"Rigidity is less than expected\" first_failing={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 121, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 100, in run_trial
    s_low_M = s_low(M)
             ^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 93, in s_low
    _, S, _ = singular_values(M)
             ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 36, in singular_valu
    norm_q = frobenius_norm(q)
             ^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 22, in frobenius_norm
    return sum(sum(x**2 for x in row) for row in M)**0.5
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfbe7700.py", line 22, in <genexpr>
    return sum(sum(x**2 for x in row) for row in M)**0.5
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError: 'int' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `frobenius_norm` (it was passed a 1D vector instead of a 2D matrix during singular value computation), so no data was produced and the pre-registered support condition could not be evaluated. | next: Fix the `singular_values/frobenius_norm` helpers to handle 1D vectors (or use `numpy.linalg.svd` and `numpy.linalg.norm`) and re-run the full sweep across `n` in `{6,8,10,12,14,16}`.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	22062
2	preregistration	claude_max	opus	0	0	7077
3	novelty	claude_max	opus	0	0	3881

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	claude_max	opus	0	0	9502
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13465
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14498
7	judge	claude_max	opus	0	0	6177

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 76660 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/223537333300.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/223537333300.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/223537333300.tar.gz (if generated)